

CareSync – Complete Project Understanding Guide (Part 1)

What is CareSync?

CareSync is a cloud-hosted healthcare appointment management platform.

The purpose of the application is to allow patients to schedule appointments with doctors, manage appointment records, receive appointment reminders, and securely store healthcare-related data.

In a real hospital or clinic, hundreds or thousands of patients may access the application simultaneously. The application must therefore be secure, scalable, highly available, fault tolerant, and easy to manage.

A simple application running on a single server is not sufficient for such a business scenario because a single server becomes a single point of failure. If that server crashes, the entire application becomes unavailable.

To solve this problem, CareSync is deployed on AWS using a modern cloud architecture.

Real Business Problem

Imagine a healthcare company that serves patients across multiple cities.

The company faces several challenges:

1. The application must always remain available.
2. Patient records must be protected.
3. Appointment reminders must be sent automatically.
4. Medical documents must be stored securely.
5. The application must handle increasing traffic.
6. Infrastructure deployment must be repeatable.
7. Engineers should not manually create resources.

Without cloud services, these requirements become difficult to achieve.

AWS provides managed services that solve these problems.

Why Cloud Computing?

Traditionally, companies purchased physical servers.

This creates problems:

- Expensive hardware purchases.
- Hardware maintenance.
- Power failures.
- Network failures.
- Difficult scaling.
- Long deployment times.

Cloud computing solves these problems.

Instead of buying servers, companies rent infrastructure from cloud providers.

Benefits include:

- Pay only for what you use.
- Rapid deployment.
- Global availability.
- Automatic scaling.
- Managed services.
- Improved security.

This is why CareSync is hosted on AWS.

Why AWS?

AWS is the largest cloud provider.

It offers:

- Compute services.
- Storage services.
- Networking services.
- Database services.
- Security services.
- Monitoring services.

AWS allows CareSync to focus on business functionality instead of infrastructure management.

Why Terraform?

AWS resources can be created manually through the AWS Console.

However, manual creation has major disadvantages.

Suppose your infrastructure contains:

- VPC
- Subnets
- Security Groups
- Load Balancers
- EC2
- RDS
- Lambda
- CloudFront
- WAF

Creating everything manually is slow and error-prone.

Terraform solves this problem.

Terraform is an Infrastructure as Code (IaC) tool.

Instead of clicking buttons in the AWS Console, we write infrastructure in code.

Benefits:

- Repeatable deployments.
- Version control.
- Collaboration.
- Automation.
- Disaster recovery.
- Consistency.

Infrastructure becomes software.

Why Terraform Modules?

Imagine writing every resource in a single file.

The file becomes huge and difficult to maintain.

Terraform Modules are reusable building blocks.

Example:

VPC resources belong inside the VPC module.

RDS resources belong inside the RDS module.

Lambda resources belong inside the Lambda module.

This creates separation of concerns.

Benefits:

- Cleaner code.
- Easier maintenance.
- Reusability.
- Better organization.
- Easier troubleshooting.

That is why CareSync is divided into modules such as:

- vpc
- security-groups
- alb
- asg
- rds
- kms
- secrets-manager
- s3
- iam
- cloudwatch
- sns
- lambda
- cloudfront
- waf

Why Remote State?

Terraform keeps track of infrastructure using a state file.

The state file contains information about every AWS resource managed by Terraform.

Without the state file Terraform cannot know:

- What already exists.
- What must be created.
- What must be updated.
- What must be destroyed.

Initially the state file was stored locally.

This created problems:

- Lost laptop = lost state.
- Team collaboration becomes difficult.
- Risk of corruption.
- No centralized source of truth.

Therefore we migrated to a remote backend.

Why S3 Backend?

Amazon S3 stores the Terraform state file.

Benefits:

- Durable.
- Highly available.
- Centralized.
- Accessible from multiple laptops.
- Backup friendly.

Current flow:

Terraform → S3 State File

Both personal laptop and company laptop use the same state file.

Therefore infrastructure remains synchronized.

Why DynamoDB Locking?

Imagine two engineers running Terraform simultaneously.

Engineer A:

terraform apply

Engineer B:

terraform apply

Both attempt to modify infrastructure.

This can corrupt state.

DynamoDB prevents this.

Terraform creates a lock before making changes.

Only one user can hold the lock.

After completion, the lock is released.

This prevents concurrent modifications.

High-Level Architecture

User



CloudFront CDN



WAF



External Application Load Balancer



Frontend Auto Scaling Group



Backend Auto Scaling Group



RDS PostgreSQL

Additional Services:

- S3 for document storage
- Secrets Manager for credentials
- KMS for encryption
- SNS for notifications
- Lambda for automation
- CloudWatch for monitoring

This architecture follows real-world enterprise cloud design principles.